การแฮชแบบขนานในเวลาจริงบนหน่วยประมวลผลกราฟิก

(Real-Time Parallel Hashing on the GPU)

บันทึกสรุปเชิงวิชาการ

เอกสารต้นทางที่สรุป:
Dan A. Alcantara, Andrei Sharf, Fatemeh Abbasinejad, Shubhabrata Sengupta, Michael Mitzenmacher, John D. Owens, and Nina Amenta. *Real-Time Parallel Hashing on the GPU*. โปสเตอร์ในงาน NVIDIA GPU Technology Conference (GTC) ปี 2009 (โปสเตอร์ระบุว่างานนี้ “To appear in SIGGRAPH Asia 2009”). University of California, Davis และ Harvard School of Engineering and Applied Sciences.

URL: `https://www.nvidia.com/content/GTC/posters/82_Alcantara_Real_Time_Parallel_Hashing.pdf`

บทความฉบับเต็มที่เกี่ยวข้อง (ใช้เป็นภูมิหลัง):
D. A. Alcantara *et al.*, “Real-Time Parallel Hashing on the GPU,” *ACM Transactions on Graphics* (Proc. SIGGRAPH Asia 2009), 28(5), 2009.

เอกสารนี้เป็น**บันทึกสรุปเพื่อการศึกษา**ที่ผู้จัดทำเรียบเรียงขึ้นใหม่ด้วยถ้อยคำของตนเอง
ไม่ใช่คำแปลของโปสเตอร์ต้นฉบับ และไม่ได้ทำซ้ำภาพหรือแผนภูมิจากต้นฉบับ
ลิขสิทธิ์ของเนื้อหาต้นฉบับเป็นของผู้เขียนโปสเตอร์

จัดทำเพื่อประกอบโครงการ CUDA Hash Table

บทคัดย่อ

บันทึกฉบับนี้สรุปและอธิบายโปสเตอร์ของ Alcantara และคณะ ซึ่งเสนอวิธีสร้างตารางแฮช (hash table) บนหน่วยประมวลผลกราฟิก (GPU) ให้เสร็จในระดับเวลาที่ใช้โต้ตอบได้ (interactive rate) สำหรับข้อมูลหลายล้านรายการ แนวคิดหลักคือการผสานสองเทคนิคเข้าด้วยกัน ได้แก่ การแบ่งข้อมูลลงถัง (bucket) ขนาดเล็กตามแนวทางของ การแฮชสมบูรณ์แบบ FKS (FKS perfect hashing) และการสร้างตารางแฮชแบบคุกคู (cuckoo hashing) แบบขนาน ภายในแต่ละถังโดยใช้หน่วยความจำบนชิป (shared memory) ผลที่ได้คือโครงสร้างที่การค้นหาแต่ละครั้งต้องตรวจสอบ ไม่เกินสามตำแหน่ง ในการทดลองกับข้อมูลวอกเซล (voxel) ราว 3.5 ล้านรายการ โปสเตอร์รายงานว่าเวลาการสร้าง ใกล้เคียงกับการเรียงลำดับแบบ radix sort ขณะที่การค้นหาแบบสุ่มเร็วกว่าการค้นหาแบบทวิภาค (binary search) บนแถวลำดับที่เรียงแล้วอย่างสม่ำเสมอ นอกจากสรุปเนื้อหาของโปสเตอร์แล้ว บันทึกนี้ได้เพิ่มภูมิหลังเรื่องการแฮช FKS การแฮชแบบคุกคู และสถาปัตยกรรมหน่วยความจำของ GPU รวมถึงข้อสังเกตเชิงปฏิบัติสำหรับการพัฒนา ตารางแฮชด้วย CUDA โดยแยกแยะให้ชัดว่าข้อความใดมาจากโปสเตอร์และข้อความใดเป็นส่วนที่ผู้จัดทำเพิ่มเติม

สารบัญ

1 บทนำ **3**

2 ภูมิหลัง **3**

2.1 ตารางแฮชและการชนกัน 3

2.2 การแฮชสมบูรณ์แบบ FKS 3

2.3 การแฮชแบบคุกคู 4

2.4 ลำดับชั้นหน่วยความจำของ GPU 4

3 ปัญหาที่โปสเตอร์ตั้งไว้ **4**

4 แนวทางที่เสนอ **5**

4.1 เหตุใดจึงใช้ FKS หรือคุกคูเพียงอย่างเดียวไม่ได้ 5

4.2 โครงสร้างสองชั้น (two-tier) 5

4.3 การแฮชแบบคุกคูแบบขนาน 5

4.4 การค้นหา 6

4.5 ส่วนขยาย 7

5 ผลการทดลอง **7**

5.1 ชุดข้อมูลและพารามิเตอร์ 7

5.2 เปรียบเทียบกับแถวลำดับที่เรียงแล้ว 7

5.3 เวลาของแต่ละขั้นตอน 8

6 การประยุกต์ใช้งาน **8**

7 ข้อสังเกตสำหรับการนำไปใช้ในโครงการ CUDA Hash Table 8

8 บทสรุป 9

1 บทนำ

ตารางแฮชเป็นโครงสร้างข้อมูลพื้นฐานสำหรับการเข้าถึงข้อมูลแบบสุ่ม (random access) ด้วยกุญแจ (key) บนหน่วยประมวลผลกลาง (CPU) ตารางแฮชถูกใช้อย่างแพร่หลาย แต่การย้ายโครงสร้างนี้ไปทำงานบน GPU ซึ่งมีเธรด (thread) ทำงานพร้อมกันนับหมื่นนั้นไม่ตรงไปตรงมา เพราะขั้นตอนการแทรก (insertion) แบบดั้งเดิม มักเป็นลำดับขั้นที่พึ่งพากัน และรูปแบบการเข้าถึงหน่วยความจำกระจัดกระจาย

โปสเตอร์ที่สรุปในบันทึกนี้นำเสนอผลงานซึ่งต่อมาตีพิมพ์เป็นบทความใน ACM Transactions on Graphics (SIGGRAPH Asia 2009) เนื่องจากโปสเตอร์มีเนื้อหากระชับมาก บันทึกนี้จึงเพิ่มคำอธิบายพื้นฐานเพื่อให้อ่านเข้าใจได้ในฐานะเอกสารวิชาการที่สมบูรณ์ในตัว

ข้อตกลงการกำกับแหล่งที่มา ตลอดเอกสารนี้ใช้ป้ายกำกับดังนี้

- **[โปสเตอร์]** เนื้อหาหรือตัวเลขที่ปรากฏในโปสเตอร์ (เรียบเรียงใหม่ ไม่ใช่การแปลตรงตัว)
- **[ภูมิหลัง]** ความรู้พื้นฐานหรือเนื้อหาจากเอกสารอื่นที่ผู้จัดทำเพิ่มเข้ามา
- **[ข้อสังเกตผู้จัดทำ]** การตีความ การคำนวณต่อยอด หรือข้อเสนอแนะของผู้จัดทำเอง

2 ภูมิหลัง

2.1 ตารางแฮชและการชนกัน

[ภูมิหลัง] ตารางแฮชใช้ฟังก์ชันแฮช (hash function) h แปลงกุญแจ k ไปเป็นดัชนีของช่อง (slot) ในแถวลำดับ เมื่อกุญแจสองตัวถูกแปลงไปยังช่องเดียวกันจะเกิด**การชนกัน** (collision) วิธีจัดการที่พบบ่อยได้แก่

- **การต่อสาย** (chaining): เก็บรายการที่ชนกันไว้ในรายการโยง (linked list) ที่ช่องนั้น
- **การระบุที่อยู่แบบเปิด** (open addressing) เช่น linear probing: หาช่องว่างถัดไปตามลำดับที่กำหนด

ทั้งสองวิธีทำให้จำนวนครั้งที่ต้องตรวจสอบหน่วยความจำ (probe) ต่อการค้นหาหนึ่งครั้งแปรผันตามข้อมูล ในกรณีเลวร้ายอาจยาวมาก

2.2 การแฮชสมบูรณ์แบบ FKS

[ภูมิหลัง] การแฮชสมบูรณ์แบบ (perfect hashing) คือการเลือกฟังก์ชันแฮชสำหรับเซตกุญแจที่รู้ล่วงหน้า ให้ไม่มีการชนกันเลย จึงค้นหาได้ในเวลา $O(1)$ ในกรณีเลวร้าย Fredman, Komlós และ Szemerédi [3] เสนอโครงสร้างสองระดับ: ระดับแรกแฮชกุญแจ n ตัวลงถัง จากนั้นแต่ละถังที่มีกุญแจ n_i ตัวใช้ตารางย่อยขนาด n_i^2 ซึ่งฟังก์ชันแฮชที่สุ่มเลือกจากตระกูลสากล (universal family) มีโอกาสไม่ชนกันอย่างน้อยครึ่งหนึ่ง ผลรวม $\sum n_i^2$ มีค่าคาดหมายเป็น $O(n)$

[โปสเตอร์] โปสเตอร์สรุปจุดเด่นและข้อจำกัดของ FKS ไว้ว่า หากใช้ตารางขนาด n^2 สำหรับกุญแจ n ตัว ฟังก์ชันแฮชแบบสุ่มมักไม่ชนกัน แต่ตารางจะโปร่งมาก (sparse) การแบ่งลงถังเล็กก่อนช่วยลดขนาดรวมเหลือ $O(n)$ ทว่ายังต้องมีถังเล็กและถังว่างจำนวนมาก จนใช้พื้นที่เกิน $3n$ ช่อง

2.3 การแฮชแบบคุกคู

[ภูมิหลัง] การแฮชแบบคุกคู (cuckoo hashing) เสนอโดย Pagh และ Rodler [4, 5] ใช้ตารางย่อยหลายตาราง แต่ละตารางมีฟังก์ชันแฮชของตนเอง กุญแจแต่ละตัวจึงมี “ที่อยู่ที่เป็นไปได้” เพียงไม่กี่ตำแหน่ง (ตำแหน่งละหนึ่งตารางย่อย) การค้นหาจึงตรวจไม่เกินจำนวนตารางย่อย เมื่อแทรกกุญแจแล้วตำแหน่งเป้าหมายถูกยึดอยู่ กุญแจใหม่จะ “ขับไล่” (evict) กุญแจเดิมออก ให้กุญแจเดิมไปหาที่อยู่ในตารางย่อยอื่น ซึ่งอาจขับไล่ต่อเป็นทอด ๆ คล้ายนกคุกคูที่ผลักไข่นกอื่นออกจากรัง หากการขับไล่วนนานเกินกำหนดจะถือว่าล้มเหลวและสร้างใหม่ด้วยฟังก์ชันแฮชชุดใหม่

[โปสเตอร์] โปสเตอร์ระบุว่าการเพิ่มจำนวนตารางย่อยช่วยให้ใช้พื้นที่ได้ดีขึ้น โดยเมื่อใช้สามตารางย่อยจะบรรจุได้สูงถึงราว 90%

2.4 ลำดับชั้นหน่วยความจำของ GPU

[ภูมิหลัง] ในสถาปัตยกรรม CUDA เธรดถูกจัดเป็นบล็อก (thread block) แต่ละบล็อกมีหน่วยความจำร่วม (shared memory) ขนาดเล็กอยู่บนชิป ซึ่งเร็วกว่าหน่วยความจำส่วนกลาง (global memory) มาก แต่เธรดต่างบล็อกกันไม่สามารถประสานจังหวะ (synchronize) กันได้ง่ายภายในเคอร์เนลเดียว การออกแบบที่ให้แต่ละบล็อกทำงานกับข้อมูลชุดเล็กของตนเองใน shared memory อย่างอิสระ จึงเป็นรูปแบบที่ได้ประสิทธิภาพดีบน GPU

3 ปัญหาที่โปสเตอร์ตั้งไว้

[โปสเตอร์] เป้าหมายคือโครงสร้างข้อมูลที่เป็นมิตรกับการประมวลผลขนาน รองรับการเข้าถึงแบบสุ่มต่อข้อมูลหลายล้านรายการ และ*สร้างเสร็จได้ในระดับเวลาที่ใช้โต้ตอบได้* โปสเตอร์ชี้ว่าเทคนิคแฮชทั่วไปขนานได้ไม่ดีด้วยเหตุผลสามประการ

ตารางที่ 1: อุปสรรคสามประการของการทำแฮชแบบขนาน (เรียบเรียงจากโปสเตอร์)

ประเด็น	คำอธิบาย
การประสานจังหวะ (synchronization)	การแทรกข้อมูลโดยทั่วไปเป็นลำดับขั้นที่ต้องทำต่อเนื่องกัน
งานต่อการเข้าถึงไม่เท่ากัน (variable work per access)	จำนวน probe ต่างกันในแต่ละคำค้น เธรดใน warp เดียวกันจึงต้องรอเธรดที่ช้าที่สุด
การเก็บแบบโปร่ง (sparse storage)	ทั้งการสร้างและการเข้าถึงมี locality ต่ำ แคชและลำดับชั้นหน่วยความจำจึงช่วยได้น้อย

[โปสเตอร์] ตารางแฮชสมบูรณ์แบบแก้ปัญหาเรื่องจำนวน probe ได้ เพราะเข้าถึงทุกรายการได้ใน $O(1)$ แต่ก่อนหน้างานนี้การสร้างตารางลักษณะดังกล่าวต้องทำบน CPU ข้อเสนอของผู้เขียนคือ การสร้างบน GPU โดยผสาน FKS กับการแฮชแบบคุกคู

[ข้อสังเกตผู้จัดทำ] ในที่นี้ “O(1)” ควรเข้าใจว่าเป็นขอบบนคงที่ของจำนวน probe ต่อคำค้น (ไม่ขึ้นกับข้อมูล) ซึ่งมีความหมายมากบน GPU เพราะลดปัญหาเธรดแตกแขนง (divergence) ภายใน warp

4 แนวทางที่เสนอ

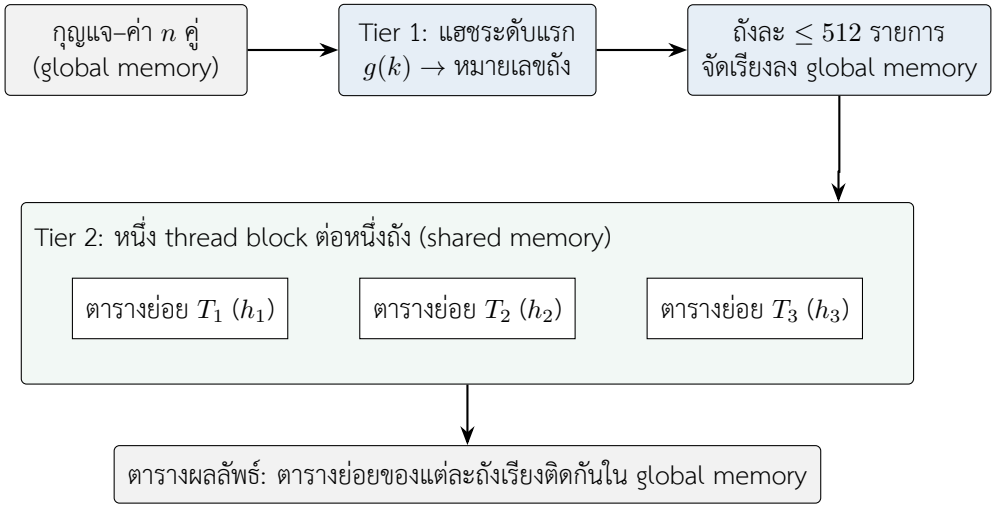
4.1 เหตุใดจึงใช้ FKS หรือคุกคูเพียงอย่างเดียวไม่ได้

[โปสเตอร์] โปสเตอร์ให้เหตุผลว่า FKS เร็วแต่กินพื้นที่มากเกินไป ส่วนการแฮชแบบคุกคูประหยัดพื้นที่ได้ แต่ถ้าแทรกแบบขนานบนตารางเดียวขนาดใหญ่ ต้องทำงานใน global memory ที่ช้าและต้องประสานจังหวะทั่วทั้งอุปกรณ์ (global synchronization)

4.2 โครงสร้างสองชั้น (two-tier)

[โปสเตอร์] วิธีที่เสนอนำจุดแข็งของทั้งสองมารวมกันเป็นสองชั้น

1. **ชั้นที่ 1 (Tier 1):** ใช้ฟังก์ชันแฮชระดับแรกแบ่งข้อมูลเข้าถังขนาดเล็ก เหมือนแนวคิดของ FKS โดยแต่ละถังมีได้ไม่เกิน 512 รายการ
2. **ชั้นที่ 2 (Tier 2):** สร้างตารางแฮชแบบคุกคูแบบขนานของแต่ละถังอย่างอิสระใน shared memory แต่ละถังประกอบด้วยตารางย่อยสามตาราง เมื่อเสร็จแล้วจึงเขียนตารางย่อยของถังเดียวกันเรียงติดกันลง global memory



ภาพที่ 1: แผนภาพโครงสร้างสองชั้นที่ผู้จัดทำวาดขึ้นใหม่จากคำอธิบายในโปสเตอร์ (ไม่ใช่ภาพจากต้นฉบับ)

[ข้อสังเกตผู้จัดทำ] ข้อดีของการแบ่งถังคือปัญหาใหญ่หนึ่งก้อนถูกแตกเป็นปัญหาเล็กจำนวนมากที่ไม่ขึ้นต่อกัน แต่ละถังพอดีกับ shared memory ของหนึ่งบล็อก การประสานจังหวะจึงเหลือเพียงระดับบล็อก ผ่านคำสั่ง `__syncthreads()` แทนการประสานทั้งอุปกรณ์ และการล้มเหลวของถังหนึ่งก็สร้างใหม่เฉพาะถังนั้นได้

4.3 การแฮชแบบคุกคูแบบขนาน

[โปสเตอร์] ผู้เขียนดัดแปลงขั้นตอนการแทรกของคุกคูให้ทำงานขนานบน GPU โดยมีหลักการดังนี้

- ทุกรายการพยายามแทรก*พร้อมกัน* และวนไปตามตารางย่อยแบบผลัดกัน (round-robin)
- เมื่อหลายรายการเขียนลงช่องเดียวกัน ถือว่า*มีการเขียนสำเร็จเพียงหนึ่งรายการพอดี* รายการที่ถูกเขียนทับต้องลองตารางย่อยถัดไปในรอบต่อไป

- สำหรับกรณีตารางย่อยสองตาราง โดยทั่วไปจบภายใน $O(\lg n)$ รอบ

[โปสเตอร์] โปสเตอร์มีตัวอย่างประกอบด้วยกุญแจสี่ตัวและตารางย่อยสองตาราง ซึ่งสรุปสาระได้ว่า ในรอบแรกทุกกุญแจเขียนลงตารางย่อยแรกพร้อมกัน มีบางตัวแพ้การชน ในรอบถัดมาตัวที่แพ้ย้ายไปตารางย่อยที่สอง และอาจชนกันเองอีก ตัวที่แพ้อีกครั้งจะวนกลับไปตารางย่อยแรกและขับไล่กุญแจที่อยู่เดิม ซึ่งกุญแจที่ถูกขับไล่ก็จะไปหาที่ในตารางย่อยที่สองต่อ จนทุกตัวมีที่อยู่

[ข้อสังเกตผู้จัดทำ] เพื่อให้เห็นกลไกชัดเจน ตารางที่ 2 เป็นตัวอย่างที่ผู้จัดทำสร้างขึ้นเอง (ไม่ใช่ตัวอย่างในโปสเตอร์) มีกุญแจ p, q, r, s ตารางย่อย T_1, T_2 ขนาดตารางละ 2 ช่อง และสมมติค่าฟังก์ชันแฮชเป็น $h_1(p) = h_1(q) = h_1(r) = 0$, $h_1(s) = 1$, $h_2(p) = h_2(r) = 0$ และ $h_2(q) = 1$ ทุกกุญแจเริ่มที่ T_1 และกุญแจที่ไม่มีที่อยู่จะลองตารางย่อยถัดจากตารางที่ตนเพิ่งอยู่หรือเพิ่งลอง

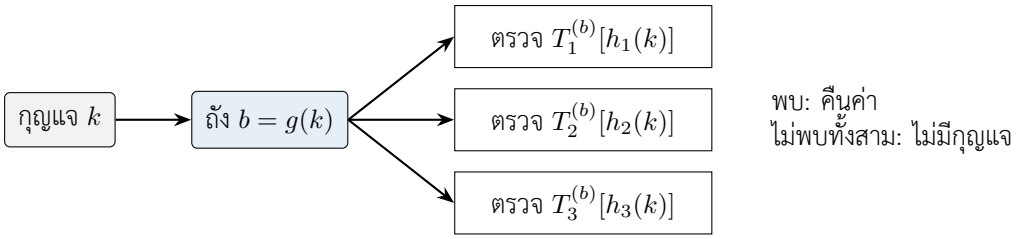
ตารางที่ 2: ตัวอย่างการแทรกแบบขนานที่ผู้จัดทำสร้างขึ้นเอง (ผู้ชนะการเขียนพร้อมกันกำหนดตามอำเภอใจ)

รอบ	การเขียนในรอบนี้	ผลลัพธ์	สถานะหลังรอบ
1	$p, q, r \to T_1[0]$; $s \to T_1[1]$	q ชนะที่ $T_1[0]$; s สำเร็จ; p, r ยังไม่มีที่อยู่	$T_1 = [q, s]$, $T_2 = [-, -]$
2	$p, r \to T_2[0]$	p ชนะ; r ยังไม่มีที่อยู่	$T_1 = [q, s]$, $T_2 = [p, -]$
3	$r \to T_1[0]$	r เขียนทับ q ทำให้ q ถูกขับไล่	$T_1 = [r, s]$, $T_2 = [p, -]$
4	$q \to T_2[1]$	สำเร็จ ทุกกุญแจมีที่อยู่	$T_1 = [r, s]$, $T_2 = [p, q]$

[ข้อสังเกตผู้จัดทำ] ประเด็นสำคัญคือการตรวจว่า “ใครชนะ” ทำได้โดยให้แต่ละเธรดเขียนแล้วอ่านกลับ หากค่าที่อ่านได้ไม่ใช่ของตนแสดงว่าแพ้ และต้องไปลองตารางย่อยถัดไป ส่วนกรณีที่การเขียนของตนไปทับกุญแจเดิม กุญแจเดิมนั้นจะกลายเป็นผู้ไม่มีที่อยู่ในรอบถัดไป (ถูกขับไล่) สำหรับ GPU รุ่นใหม่ คำสั่งอะตอมมิก (atomic operation) ประเภทแลกเปลี่ยนค่า เช่น `atomicExch` จะคืนค่าที่ถูกแทนที่กลับมาโดยตรง

4.4 การค้นหา

[โปสเตอร์] เมื่อสร้างเสร็จ การค้นหากุญแจใช้ probe ไม่เกินสามครั้ง คือหาถังด้วยฟังก์ชันแฮชระดับแรก แล้วตรวจหนึ่งช่องในแต่ละตารางย่อยทั้งสามของถังนั้น โปสเตอร์ยกตัวอย่างว่าโครงสร้างสำหรับข้อมูลทดสอบมีราว 5 ล้านช่อง แต่การหารายการใดก็ตามต้องดูเพียงสามตำแหน่ง



ภาพที่ 2: ขั้นตอนการค้นหาที่มีจำนวน probe คงที่ (ผู้จัดทำวาดเองจากคำอธิบายในโปสเตอร์)

4.5 ส่วนขยาย

[โปสเตอร์] โปสเตอร์ระบุว่าอัลกอริทึมสามารถขยายให้รองรับ*หลายค่าต่อหนึ่งกุญแจ* (multiple values per key) และใช้สร้าง*ดัชนีสองทาง* (two-way index) ระหว่างกุญแจกับหมายเลขประจำตัวที่ไม่ซ้ำ (unique ID) โปสเตอร์ไม่ได้ให้รายละเอียดวิธีการของส่วนขยายนี้ ผู้สนใจควรศึกษาจากบทความฉบับเต็ม [2]

5 ผลการทดลอง

5.1 ชุดข้อมูลและพารามิเตอร์

[โปสเตอร์] ข้อมูลตัวอย่างหลักได้จากการแปลงแบบจำลองสามมิติ “Lucy” เป็นวอกเซล ได้ราว 3.5 ล้านวอกเซล เก็บเป็นกุญแจขนาด 32 บิต ตัวเลขที่โปสเตอร์ระบุเป็นข้อความสรุปไว้ในตารางที่ 3

ตารางที่ 3: ตัวเลขที่โปสเตอร์ระบุเป็นข้อความ และค่าที่ผู้จัดทำคำนวณต่อ

รายการ	ค่า	แหล่งที่มา
จำนวนกุญแจในตัวอย่าง Lucy	≈ 3.5 ล้าน	[โปสเตอร์]
ขนาดกุญแจ	32 บิต	[โปสเตอร์]
จำนวนช่องทั้งหมดของโครงสร้าง	≈ 5 ล้าน	[โปสเตอร์]
ความจุสูงสุดต่อถัง	512 รายการ	[โปสเตอร์]
จำนวนรายการเฉลี่ยต่อถัง	409	[โปสเตอร์]
จำนวนตารางย่อยต่อถัง	3	[โปสเตอร์]
จำนวน probe สูงสุดต่อการค้นหา	3	[โปสเตอร์]
อัตราการบรรจุถังเฉลี่ย $409/512$	$\approx 80\%$	[ข้อสังเกตผู้จัดทำ]
อัตราการบรรจุตารางรวม $3.5/5$	$\approx 70\%$	[ข้อสังเกตผู้จัดทำ]
จำนวนช่องต่อกุญแจ $5/3.5$	≈ 1.4	[ข้อสังเกตผู้จัดทำ]

[ข้อสังเกตผู้จัดทำ] ค่า ≈ 1.4 ช่องต่อกุญแจนี้น้อยกว่าพื้นที่มากกว่า $3n$ ของ FKS ที่โปสเตอร์กล่าวถึงอย่างชัดเจน และสอดคล้องกับข้ออ้างว่าการรวมคุกคูเข้ามาช่วยลดพื้นที่ (ทั้งนี้ตัวเลข 5 ล้านเป็นค่าโดยประมาณจากโปสเตอร์)

5.2 เปรียบเทียบกับแถวลำดับที่เรียงแล้ว

[โปสเตอร์] แผนภูมิแรกในโปสเตอร์วัดเวลา (มิลลิวินาที) เมื่อเพิ่มความละเอียดของการแปลง Lucy เป็นวอกเซล จนมีคู่กุญแจ–ค่าราว 3.5 ล้านคู่ โดยเทียบวิธีที่เสนอกับแถวลำดับที่เรียงด้วย radix sort แล้วค้นด้วย binary search และมีเส้นของการค้นหาด้วย PSH บน CPU ด้วย ข้อสรุปเชิงคุณภาพที่โปสเตอร์เขียนไว้คือ

- เวลาสร้างตารางแฮชบน GPU ใกล้เคียงกับเวลาของ radix sort
- การค้นหาแบบสุ่มด้วยตารางแฮชเร็วกว่า binary search อย่างสม่ำเสมอ

[ข้อสังเกตผู้จัดทำ] จากการอ่านค่าจากแผนภูมิด้วยสายตา (จึงเป็นค่าประมาณคร่าว ๆ ไม่ใช่ตัวเลขที่ผู้เขียนรายงาน) ที่ขนาดราว 3.5 ล้านคู่ เวลาสร้างและเวลา radix sort อยู่ในช่วงราว 25–30 มิลลิวินาที เวลาค้นหาด้วยตารางแฮชอยู่ราว 10 มิลลิวินาที ขณะที่ binary search ใช้ราว 55 มิลลิวินาที กล่าวคือการค้นหาเร็วกว่าราวห้าเท่าในจุดนั้น ผู้อ่านที่ต้องการตัวเลขแม่นยำควรอ้างอิงบทความฉบับเต็ม

[ภูมิหลัง] PSH ในที่นี้คือ Perfect Spatial Hashing ของ Lefebvre และ Hoppe [6] ซึ่งเป็นตารางแฮชสมบูรณ์แบบสำหรับข้อมูลเชิงพื้นที่ที่ต้องสร้างบน CPU (ใช้เวลานาน) แต่ค้นหาบน GPU ได้เร็ว

5.3 เวลาของแต่ละขั้นตอน

[โปสเตอร์] แผนภูมิที่สองแสดงเวลาของแต่ละขั้นตอนเมื่อจำนวนกุญแจสุ่มเพิ่มขึ้นถึง 10 ล้านคู่ ขั้นตอนที่แยกให้เห็นได้แก่ (1) การกำหนดค่าเริ่มต้น (initialization) (2) การกำหนดถังให้กุญแจและนับจำนวนต่อถัง (3) การหาตำแหน่งข้อมูลของแต่ละถัง (4) การย้ายข้อมูลเข้าถัง (shuffling) และ (5) การแฮชแบบคุกคู โปสเตอร์สรุปว่าทุกขั้นตอนเพิ่มขึ้นเป็นเชิงเส้นโดยประมาณตามขนาดข้อมูล

[ข้อสังเกตผู้จัดทำ] จากการอ่านแผนภูมิด้วยสายตา ขั้นตอนการแฮชแบบคุกคูและขั้นตอนการกำหนดถังพร้อมนับเป็นสองส่วนที่ใช้เวลามากที่สุด ตามด้วยการย้ายข้อมูลเข้าถัง ส่วนการกำหนดค่าเริ่มต้นและการหาตำแหน่งถังใช้เวลาน้อยมาก ข้อสังเกตนี้บอกว่าขั้นตอนที่ใช้ atomic operation และการเข้าถึง global memory แบบกระจัดกระจาย (การนับและการย้ายข้อมูล) มีน้ำหนักพอ ๆ กับการสร้างตารางคุกคูเอง

6 การประยุกต์ใช้งาน

[โปสเตอร์] โปสเตอร์กล่าวถึงตัวอย่างการใช้งานบน GPU สองแบบที่สาธิตในบทความ

- **Geometric hashing:** ค้นหาตำแหน่งของภาพแม่แบบภายในภาพที่ใหญ่กว่า
- **Spatial hashing:** หาส่วนตัดแบบบูลีน (boolean intersection) ระหว่างกลุ่มจุด (point cloud) สองชุดที่เคลื่อนที่ โดยผู้ใช้ปรับการแปลงตำแหน่ง (transformation) ได้แบบโต้ตอบ

[ภูมิหลัง] Geometric hashing เป็นเทคนิคการรู้จำวัตถุที่เก็บคุณลักษณะเชิงเรขาคณิตของแม่แบบลงตารางแฮช แล้วให้คุณลักษณะจากภาพเป้าหมายลงคะแนน (vote) ผ่านการค้นตาราง [8] ทั้งสองการประยุกต์มีจุดร่วมคือต้องสร้างตารางใหม่บ่อยและค้นหาจำนวนมากต่อเฟรม ซึ่งตรงกับจุดแข็งของวิธีนี้ที่สร้างได้เร็วระดับหลายสิบมิลลิวินาที

7 ข้อสังเกตสำหรับการนำไปใช้ในโครงการ CUDA Hash Table

หัวข้อนี้ทั้งหมดเป็น **[ข้อสังเกตผู้จัดทำ]** หรือ **[ภูมิหลัง]** ไม่ใช่ข้อความจากโปสเตอร์

1. **เลือกขนาดถังให้พอดีกับ shared memory** หากเก็บคู่กุญแจ–ค่า 32+32 บิตเป็นคำ 64 บิต ถังที่มีราว 1.4 เท่าของ 512 ช่อง จะใช้ shared memory ราว 6 KB ต่อบล็อก ซึ่งพอให้หลายบล็อกอยู่บน SM เดียวกันได้ ค่าคงที่ที่แน่นอน (ขนาดตารางย่อย จำนวนรอบสูงสุด) ควรตรวจสอบจากบทความฉบับเต็ม [2] แทนการเดา
2. **จัดการถังล้น (bucket overflow)** ฟังก์ชันแฮชระดับแรกอาจทำให้บางถังเกิน 512 รายการ ต้องตรวจหลังขั้นตอนนับ และสุ่มฟังก์ชันระดับแรกใหม่หากล้น การเลือกจำนวนถังให้ค่าเฉลี่ยต่ำกว่าความจุพอสมควร (โปสเตอร์ใช้ค่าเฉลี่ย 409 จาก 512) ช่วยให้ความน่าจะเป็นที่ต้องเริ่มใหม่ต่ำ
3. **จัดการการแทรกล้มเหลวในระดับถัง** กำหนดจำนวนรอบสูงสุดของลูปคุกคู หากเกินให้สุ่มชุดฟังก์ชันแฮชใหม่และสร้างเฉพาะถังนั้นใหม่ ซึ่งถูกกว่าการสร้างทั้งตารางมาก

4. **กลไกตัดสินผู้ชนะ** โปสเตอร์อาศัยสมมติฐานว่าการเขียนพร้อมกันจะสำเร็จเพียงหนึ่งราย ในโค้ดจริงควรใช้ `atomicExch` หรือ `atomicCAS` บน shared memory เพื่อให้ได้ค่าที่ถูกแทนที่กลับมาชัดเจน และควรใช้คำ 64 บิตเดียวสำหรับกุญแจและค่า เพื่อไม่ให้กุญแจกับค่าหลุดจากกัน

5. **ค่าว่าง (sentinel) และกุญแจต้องห้าม** ต้องสงวนค่ากุญแจหนึ่งค่า (เช่น `0xFFFFFFFF`) แทนช่องว่าง และตรวจว่าข้อมูลจริงไม่มีค่านี้

6. **ฟังก์ชันแฮช** ตระกูลแฮชแบบสากลที่คำนวณถูก เช่น $h(k) = ((ak + b) \bmod p) \bmod m$ โดย p เป็นจำนวนเฉพาะ เหมาะกับ GPU เพราะสุ่มชุดใหม่ได้ง่ายเมื่อต้องสร้างใหม่ แต่ควรระวังการล้นของจำนวนเต็มและคุณภาพของการกระจาย

7. **การค้นหาที่มีขอบเขตคงที่** เนื่องจาก probe ไม่เกินสามครั้ง เคอร์เนลค้นหาจึงเขียนแบบไม่มีลูปไม่จำกัด (unrolled) ได้ ลด divergence ภายใน warp และควรเรียงตารางย่อยของถังเดียวกันให้อยู่ติดกัน ตามที่โปสเตอร์กล่าว เพื่อให้ probe ทั้งสามอยู่ในบริเวณหน่วยความจำใกล้กัน

8. **ข้อจำกัดของโครงสร้าง** วิธีนี้ออกแบบเป็นการสร้างแบบกลุ่ม (bulk build) จากเซตกุญแจที่รู้ล่วงหน้า การแทรกหรือลบทีละรายการแบบพลวัตไม่ใช่จุดประสงค์หลัก ถ้าโครงการต้องการตารางแบบพลวัต ควรพิจารณาการออกแบบอื่นเพิ่มเติม

9. **ทางเลือกรุ่นถัดมา** **[ภูมิหลัง]** งานต่อเนื่องของกลุ่มผู้เขียนเดียวกัน [7] เสนอการแฮชแบบคุกคูชั้นเดียว ใน global memory โดยใช้ atomic exchange และฟังก์ชันแฮชมากกว่าสามตัว ซึ่งเขียนโค้ดง่ายกว่าแบบสองชั้น ควรนำมาเปรียบเทียบเป็นค่าฐาน (baseline) ด้วย

10. **การวัดผล** ควรวัดเทียบกับ radix sort + binary search เช่นเดียวกับโปสเตอร์ แยกเวลาสร้างกับเวลาค้นหา และแยกเวลาแต่ละขั้นตอน รวมทั้งรายงานอัตราการบรรจุ (load factor) และจำนวนครั้งที่ต้องสร้างใหม่

8 บทสรุป

โปสเตอร์ของ Alcantara และคณะแสดงให้เห็นว่าการสร้างตารางแฮชที่ค้นหาได้ด้วยจำนวน probe คงที่ สามารถทำบน GPU ได้ในระดับเวลาโต้ตอบ โดยอาศัยการแบ่งปัญหาเป็นสองชั้น: ชั้นแรกแบ่งข้อมูลลงถังเล็กตามแนว FKS ชั้นที่สองสร้างตารางคุกคูสามตารางย่อยแบบขนานใน shared memory ของแต่ละถัง ผลการทดลองในโปสเตอร์ชี้ว่าเวลาสร้างเทียบเท่า radix sort การค้นหาแบบสุ่มเร็วกว่า binary search และทุกขั้นตอนขยายตัวแบบเชิงเส้นโดยประมาณ แนวคิดสำคัญที่นำไปใช้กับโครงการ CUDA ได้โดยตรงคือ การแบ่งงานให้พอดีกับหน่วยความจำบนชิป การจำกัดจำนวน probe ให้คงที่ และการออกแบบให้การล้มเหลว กู้คืนได้ในระดับถัง

เอกสารอ้างอิง

[1] D. A. Alcantara, A. Sharf, F. Abbasinejad, S. Sengupta, M. Mitzenmacher, J. D. Owens, and N. Amenta. Real-Time Parallel Hashing on the GPU. Poster, NVIDIA GPU Technology Conference (GTC), 2009. `https://www.nvidia.com/content/GTC/posters/82_Alcantara_Real_Time_Parallel_Hashing.pdf`

[2] D. A. Alcantara, A. Sharf, F. Abbasinejad, S. Sengupta, M. Mitzenmacher, J. D. Owens, and N. Amenta. Real-Time Parallel Hashing on the GPU. *ACM Transactions on Graphics* (Proc. SIGGRAPH Asia), 28(5), 2009. doi:10.1145/1618452.1618500

[3] M. L. Fredman, J. Komlós, and E. Szemerédi. Storing a sparse table with $O(1)$ worst case access time. *Journal of the ACM*, 31(3):538–544, 1984. (โปสเตอร์พิมพ์เลขหน้าเป็น “583–544” ซึ่งน่าจะเป็นการพิมพ์ผิด)

[4] R. Pagh and F. F. Rodler. Cuckoo hashing. In *Proc. 9th Annual European Symposium on Algorithms (ESA)*, LNCS vol. 2161, pp. 121–133. Springer, 2001.

[5] R. Pagh and F. F. Rodler. Cuckoo hashing. *Journal of Algorithms*, 51(2):122–144, 2004.

[6] S. Lefebvre and H. Hoppe. Perfect spatial hashing. *ACM Transactions on Graphics* (Proc. SIGGRAPH), 25(3):579–588, 2006.

[7] D. A. Alcantara, V. Volkov, S. Sengupta, M. Mitzenmacher, J. D. Owens, and N. Amenta. Building an efficient hash table on the GPU. In W. W. Hwu, editor, *GPU Computing Gems Jade Edition*, chapter 4. Morgan Kaufmann, 2011.

[8] H. J. Wolfson and I. Rigoutsos. Geometric hashing: an overview. *IEEE Computational Science and Engineering*, 4(4):10–21, 1997.